

AAEC 4484: Applied Economic Forecasting

EXAM 2 – Spring 2026

Alex Fuentes

Due: Thursday, May 7, 2026 (11:59 AM)

Instructions (PLEASE READ):

1. This is an open-book exam. You are free to use your notes, homework, and any available resource to aid with your answers. **However, you are not allowed to collaborate with anyone else (enrolled in the course or otherwise).**
 2. Where necessary, please ensure that your graphs and visuals have proper titles and axes labels.
 3. Please refer to the output, whenever appropriate, when discussing the results.
-

Question 1: Electricity Generation and Distribution [Total: 50 points]

Mr. Fuentes has just landed his first internship with Statistics South Africa and has been tasked with producing the “Electricity Generated and Available for Distribution” series ([here is a copy for February 2026](#)). The data collection desk has sent him the data and he is quickly realizing that data in the real world is not as clean as the data we have been working with in class.

In the questions that follow, we will work through a complete workflow of a real-world time series analysis — from raw, unstructured data to cleaned `tsibble`, visual diagnostics, decomposition, model adequacy tests, and forecasting.

Mr. Fuentes opens the data file (“Electricity_SA.csv” in the **Data** folder on GitHub) and sees that the data is stored in a non-standard (wide) format, with months labeled in columns such as M0012020 and M0022020, for January 2020 and February 2020, respectively. He also notices that the data is not in a `tsibble` format, which is required for time series analysis in the `fpp3` package.

- a. [5 points] Mr. Fuentes has enlisted your help to clean the data in order to work with the **Electricity available for distribution in South Africa** (under H05) and the **National Electricity Provider** (under H04). This will correspond to the code **ELEKTR20** under H03.

You looked through his email and offered the following suggestions to help him get started.

In a single step:

- (i) Read the data into **R** **directly** from GitHub.
- (ii) Filter the data to include only the relevant row under the H03 column.
- (iii) Use the `select()` function to keep the H03 column and all columns that `starts_with()` the string M0.
- (iv) Use the `pivot_longer()` function to convert the data from wide to long format. Use the `names_to = xxx` and `values_to = xxx` arguments to set the new column names to be **Date** and **Electricity**. **Hint: You will find it best to pivot on all columns *except* H03.**
- (v) Now, you can drop the column H03. You should be left with only two columns: **Date** and **Electricity**.
- (vi) The **Date** column is currently in a character format: M0012020. You need to convert it to a date format. **Note that the first 2 characters (M0) are not needed and the remaining characters (3–4 are the months, and 5–8 are the year).**

- Use my codes below to achieve this:

```
mutate(Date = yearmonth(paste0(substr(Date, 5, 8), "-",
                               substr(Date, 3, 4))))
```

Note: For completeness, we use the `paste0()` function to concatenate the year and month, separated by a `-`. The `yearmonth()` function will then convert this into a date format.

(vii) Now declare as a `tsibble()` object with the appropriate index and your job is done.

```
url1 <- "https://raw.githubusercontent.com/Shamar-Stewart/Forecasting-S26/refs/heads/main/Data/Electricity_9

electricity_ts <- read_csv(url1) %>%
  filter(H03 == "ELEKTR20") %>%
  dplyr::select(H03, starts_with("MO")) %>% # Explicitly use dplyr
  pivot_longer(cols = -H03, names_to = "Date", values_to = "Electricity") %>%
  dplyr::select(-H03) %>% # Explicitly use dplyr
  mutate(Date = yearmonth(paste0(substr(Date, 5, 8), "-", substr(Date, 3, 4)))) %>%
  as_tsibble(index = Date)

print(electricity_ts)
```

```
## # A tsibble: 314 x 2 [1M]
##       Date Electricity
##       <mtch>      <dbl>
## 1 2000 Jan      14787
## 2 2000 Feb      14521
## 3 2000 Mar      15449
## 4 2000 Apr      14632
## 5 2000 May      16270
## 6 2000 Jun      16034
## 7 2000 Jul      16973
## 8 2000 Aug      16394
## 9 2000 Sep      15448
## 10 2000 Oct     15908
## # i 304 more rows
```

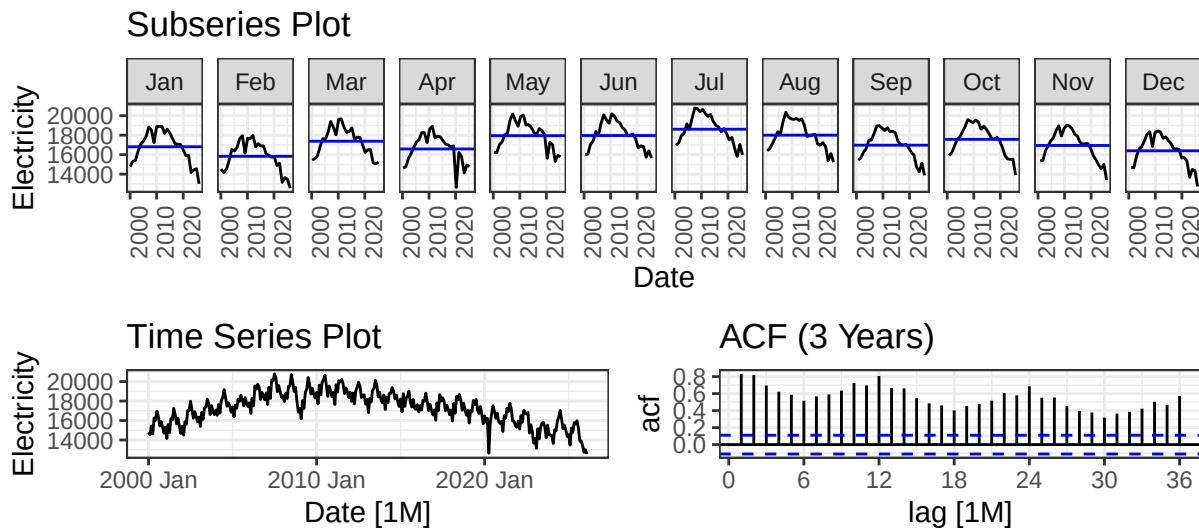
- b. [10 points] Mr. Fuentes has now cleaned the data and is ready to start his analysis. He is so impressed by the help you provided earlier that he still wants to ask you for more help. (sigh)

Tasks:

- (i) Using the `patchwork` package, produce a combined plot of the `autoplot()`, `gg_subseries()`, and `ACF()` plots (with a maximum lag of 3 years). **Note:** I would like your `gg_subseries()` plot to span the entire first row and the `ACF()` and `autoplot()` will share the second row. You might need to adjust the `fig.height` here so it is legible.

```
p1 <- electricity_ts %>% autoplot(Electricity) + labs(title = "Time Series Plot")
p2 <- electricity_ts %>% gg_subseries(Electricity) + labs(title = "Subseries Plot")
p3 <- electricity_ts %>% ACF(Electricity, lag_max = 36) %>% autoplot() + labs(title = "ACF (3 Years)")

combined_plot <- p2 / (p1 | p3)
combined_plot
```



- (ii) Comment on your observations from **each plot**. In particular, discuss any observable seasonal patterns, trends, and any other interesting features you notice.

For the ACF you can see that every lag is outside of the confidence interval especially at annual seasonal lags of 12, 24, and 36. Leading me to think that this is not white noise and we can extrapolate more information out of this process. The time series plot has a seasonal pattern with constant variance except around 2009 and after 2020 where there are many dramatic variances. It also begins with an upward trend until 2010 where a strong negative trend takes over. The sub series exhibits a pattern of higher energy usage from May to August compared to other months. They also follow the same shape of the time series plots too confirming trend.

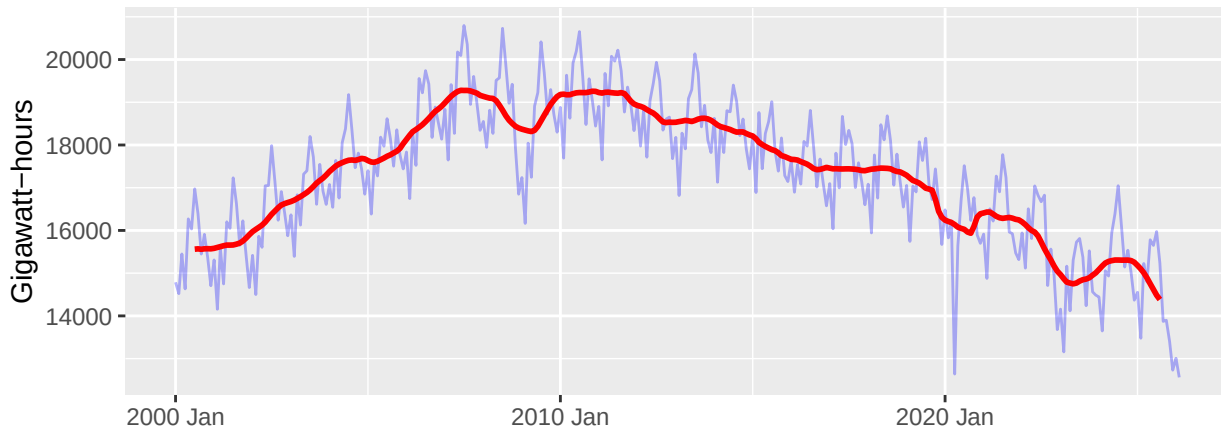
- c. [5 points] Mr. Fuentes's supervisor just stopped by his desk and asked that he produce a graph similar to Figure 1 from the report from December 2023 (**but using your current data**). You are already emotionally invested in this project and want to help him out. Using an appropriate centered moving average, depict the `trend-cycle()` component of the data above. **Hint: Be sure to pay attention to the data frequency and the fact that we need the smoothing to be symmetric on both sides.**

```
electricity_trend <- electricity_ts %>%
mutate(`12-MA` = slide_dbl(Electricity, mean, .before = 5, .after = 6, .complete = TRUE),
trend_cycle = slide_dbl(`12-MA`, mean, .before = 1, .after = 0, .complete = TRUE))

electricity_trend %>%
ggplot(aes(x = Date)) +
geom_line(aes(y = Electricity), color = "blue", alpha = 0.3) +
geom_line(aes(y = trend_cycle), color = "red", size = 1) +
labs(title = "Figure 1 - Electricity generated in South Africa",
subtitle = "Blue line: Original Series | Red line: Trend-Cycle",
y = "Gigawatt-hours",
x = "") + theme_gray()
```

Figure 1 – Electricity generated in South Africa

Blue line: Original Series | Red line: Trend-Cycle



- d. [10 points] Mr. Fuentes feels ready to take it from here and to utilize his time series forecasting skills. He recalls that we had explored a number of decomposition methods in class and wants to apply a few to his data.

Tasks:

- (i) Split the data into a training (`elec.train`) and test set (`elec.test`), reserving the last 3 years of data for out-of-sample analysis. **You would find it prudent to produce a plot of the training and test data to ensure that the split was done correctly.**

```
split_date <- max(electricity_ts$Date) - 35

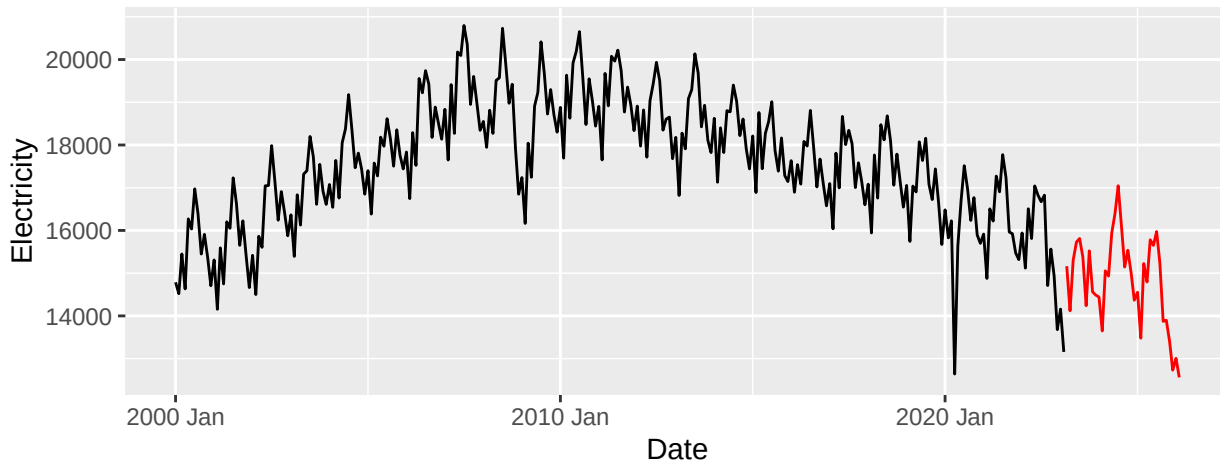
elec.train <- electricity_ts %>%
  filter(Date < split_date)

elec.test <- electricity_ts %>%
  filter(Date >= split_date)

elec.train %>%
  autoplot(Electricity) +
  autolayer(elec.test, Electricity, color = "red") +
  labs(title = "Electricity Training vs Test Set Split",
       subtitle = "Black: Training Data | Red: Test Data ",
       y = "Electricity",
       x = "Date") + theme_gray()
```

Electricity Training vs Test Set Split

Black: Training Data | Red: Test Data



(ii) Implement a classical decomposition over the training period assuming additive seasonality.

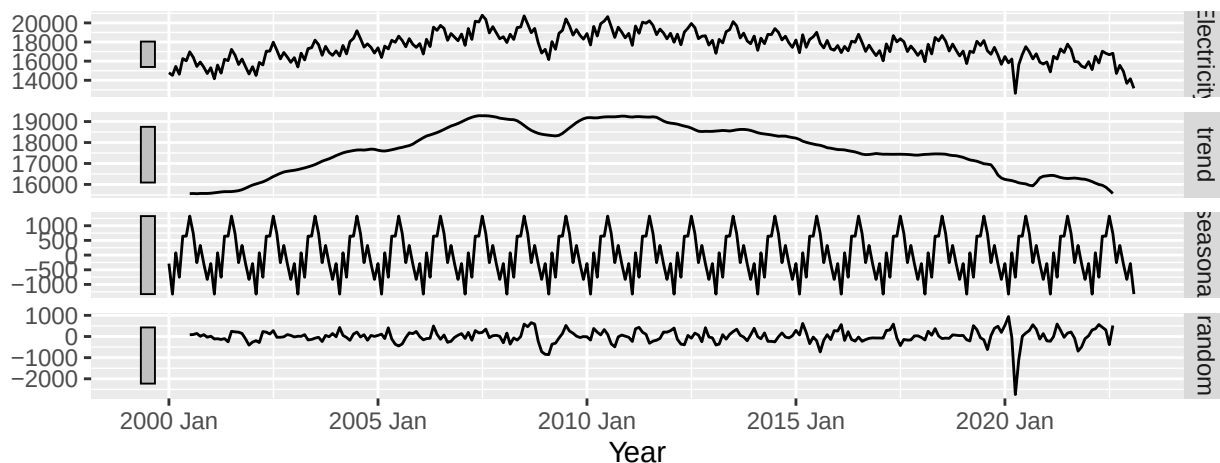
```
elec_decomp_fit <- elec.train %>%
model(classical = classical_decomposition(Electricity, type = "additive"))
```

(iii) Produce an autoplot() of the components of the decomposition.

```
elec_decomp_fit %>%
components() %>%
autoplot() + labs(title = "Classic Additive Decomposition",
subtitle = "South African Electricity Generation",
x = "Year") + theme_gray()
```

Classic Additive Decomposition

South African Electricity Generation



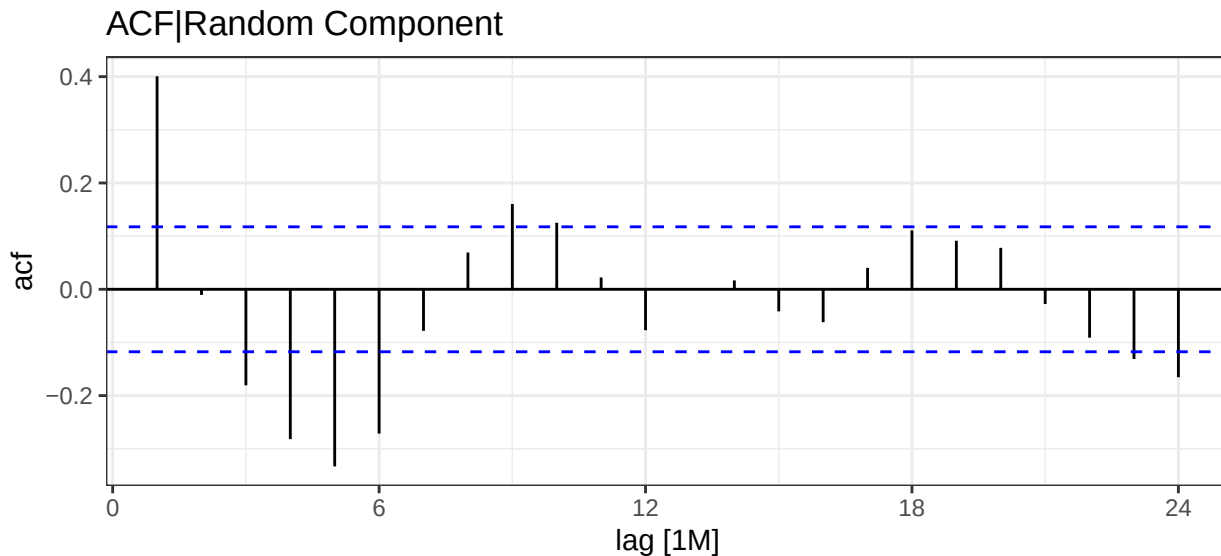
(iv) Produce an ACF() plot of the random component of this model. Do the residuals appear to be white noise? Accompany your answer with a formal hypothesis test (i.e., Ljung-Box test) and clearly state (i) the null and alternative hypotheses of the test in your discussion, and (ii) your conclusion.

I would say they do not appear to be white noise as there are 7 lags that are spiked out of the confidence interval. We are going to evaluate the residuals at the .05 alpha level and with a p-value of 0 being lower than the alpha value,

we therefore must reject the null hypothesis. There is enough evidence to conclude that the residuals are not a white noise process.

N0- The residuals are white noise N1- The residuals are not white noise

```
elec_decomp_fit %>%
  components() %>%
  ACF(random) %>%
  autoplot() + labs(title = "ACF|Random Component")
```



```
elec_decomp_fit %>%
  components() %>%
  features(random, ljung_box, lag = 24)
```

```
## # A tibble: 1 x 3
##   .model    lb_stat lb_pvalue
##   <chr>      <dbl>   <dbl>
## 1 classical    166.       0
```

- e. [5 points] Repeat steps IV(ii) – (iv) from above, but this time use the x11 decomposition method. **Reminder: The random component of the x11 decomposition is called the irregular component.**

This is a big improvement over the previous decomposition but I would say that I believe that this is not a white noise process as there are two lags that spike out of the confidence interval. Evaluating at the same alpha level as before, the p-value is still lower than .05 and we decide to reject the null hypothesis again. Concluding, that the irregular component is not a white noise process and we can still gather information out of it.

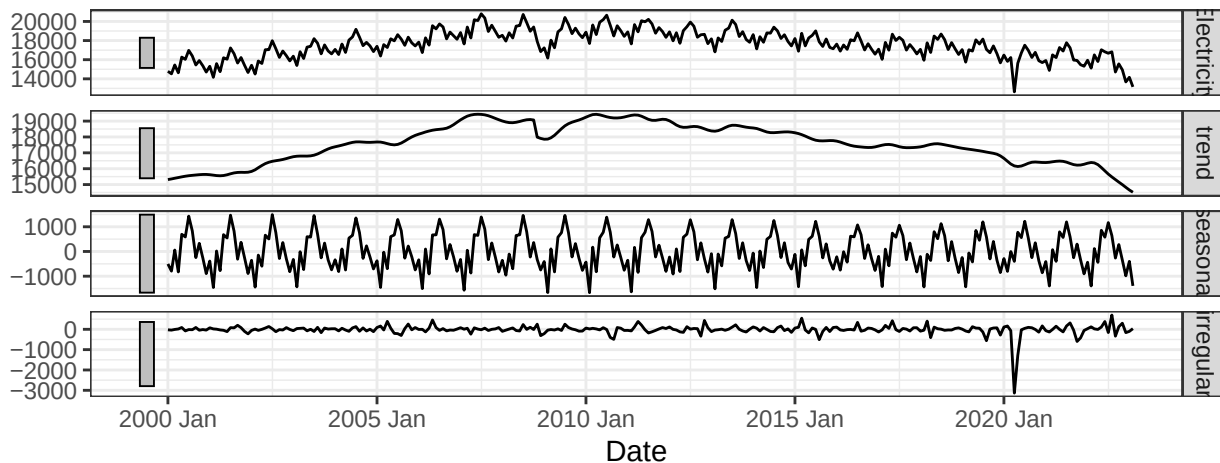
N0- The irregular components are white noise N1- The irregular components not white noise

```
elec_x11_fit <- elec.train %>%
  model(x11 = X_13ARIMA_SEATS(Electricity ~ x11()))

elec_x11_fit %>%
  components() %>%
  autoplot() + labs(title = "X11 Decomposition of Training Data")
```

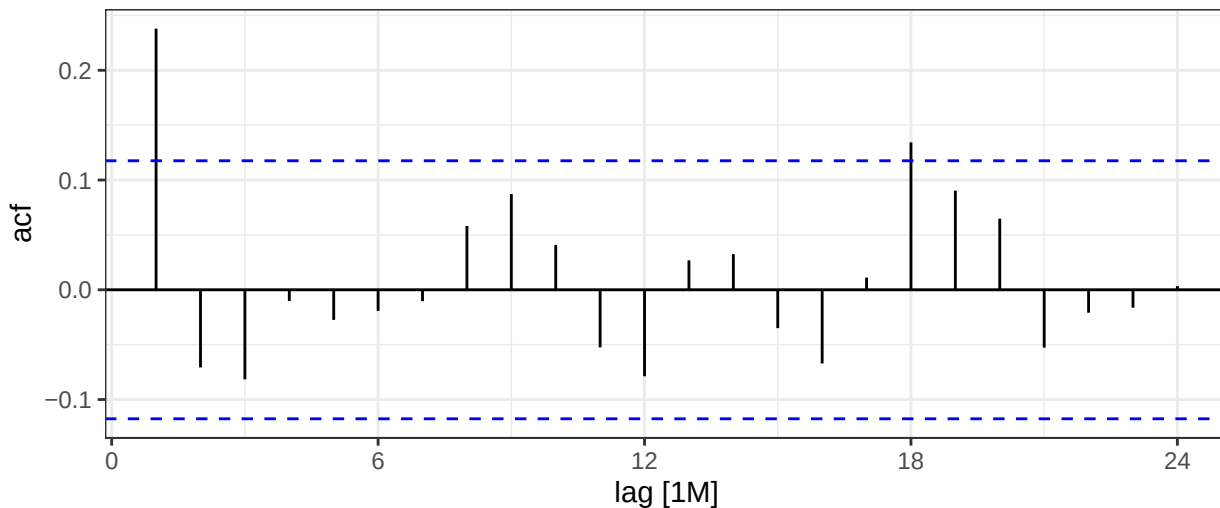
X11 Decomposition of Training Data

Electricity = trend + seasonal + irregular



```
elec_x11_fit %>%
components() %>%
ACF(irregular) %>%
autoplot() + labs(title = "ACF of the Irregular Component (X11)")
```

ACF of the Irregular Component (X11)



```
elec_x11_fit %>%
components() %>% features(irregular, ljung_box, lag = 24)
```

```
## # A tibble: 1 x 3
##   .model lb_stat lb_pvalue
##   <chr>    <dbl>    <dbl>
## 1 x11      38.3    0.0323
```

f. [5 points] Again, repeat the steps above, only this time, use the SEATS decomposition method.

There is only 1 lag spike out of the interval and at the very start, so I would proclaim that this is a white noise process. Keeping our same evaluation criteria over the past test, we evaluate the p-value of 0.691 to our alpha value of 0.05 and it is much higher, we can now fail to reject the null hypothesis and there is enough evidence to say that the irregular components are white noise.

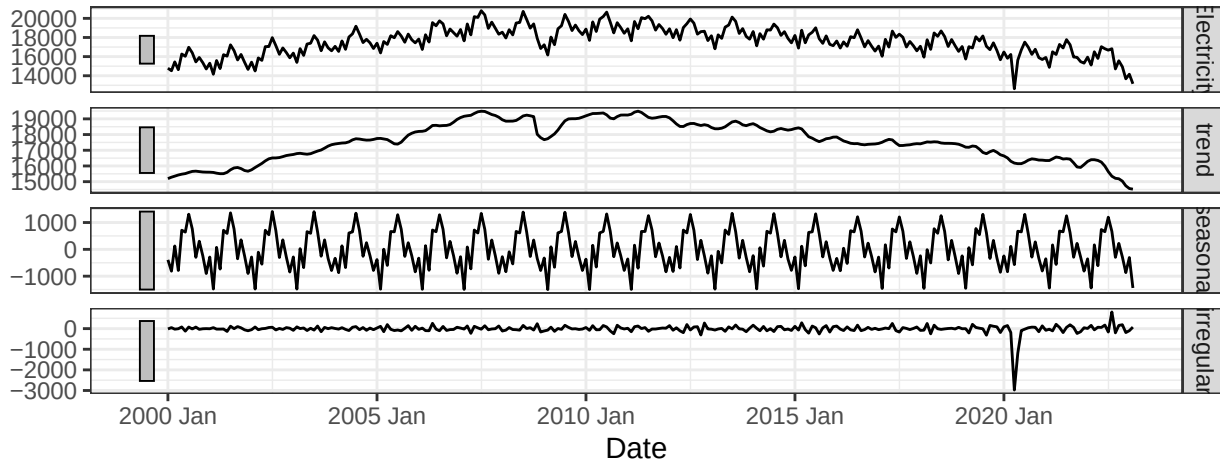
N0- The irregular components are white noise N1- The irregular components are not white noise

```
elec_seats_fit <- elec.train %>%
model(seats = X_13ARIMA_SEATS(Electricity ~ seats()))

elec_seats_fit %>%
components() %>%
autoplot() + labs(title = "SEATS Decomposition of Training Data")
```

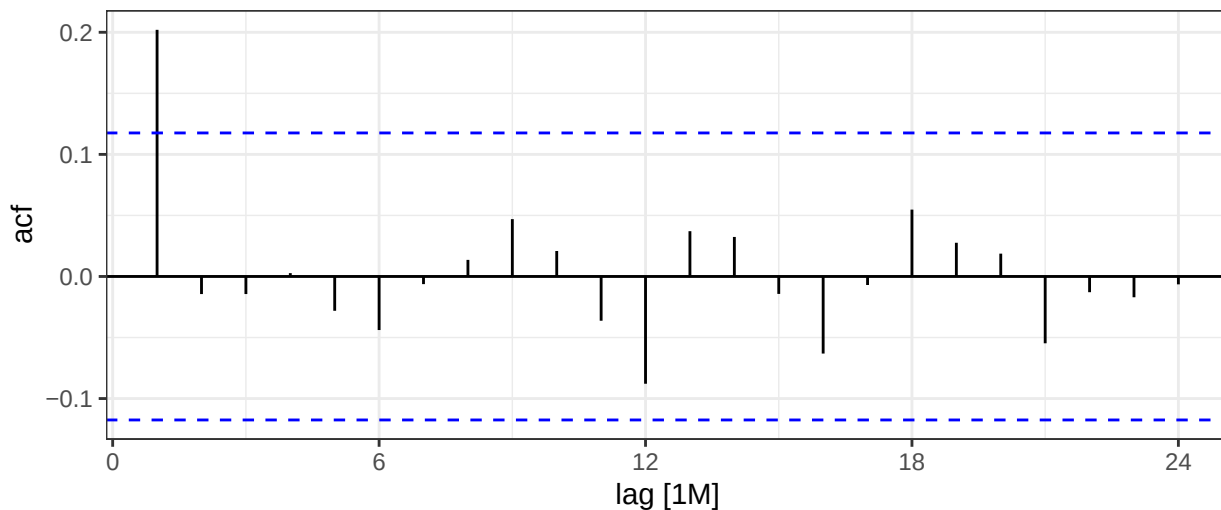
SEATS Decomposition of Training Data

Electricity = f(trend, seasonal, irregular)



```
elec_seats_fit %>%
components() %>%
ACF(irregular) %>%
autoplot() +
labs(title = "ACF of the Irregular Component (SEATS)")
```

ACF of the Irregular Component (SEATS)



```
elec_seats_fit %>%
components() %>%
features(irregular, ljung_box, lag = 24)
```

```
## # A tibble: 1 x 3
##   .model lb_stat lb_pvalue
```



```
##      <chr>      <dbl>      <dbl>
## 1 seats        20.1        0.691
```

- g. [5 points] His last candidate model is a STL decomposition method that is **robust** to outliers, has a **trend window** of 7 and **seasonal window** that is **periodic**. Again, repeat the analysis from the previous step.

I would again say that his model is white noise due to there only being two significant spikes at lag 1 and 2. They decompose fast and go below the confidence bands quickly leading me to my prediction. Using the same decision ruling and alpha criteria level as before. The p-value of 0.0984 is higher than 0.05 and we conclude that we reject the null hypothesis. Thus, there is enough evidence to conclude that the remainder component is white noise.

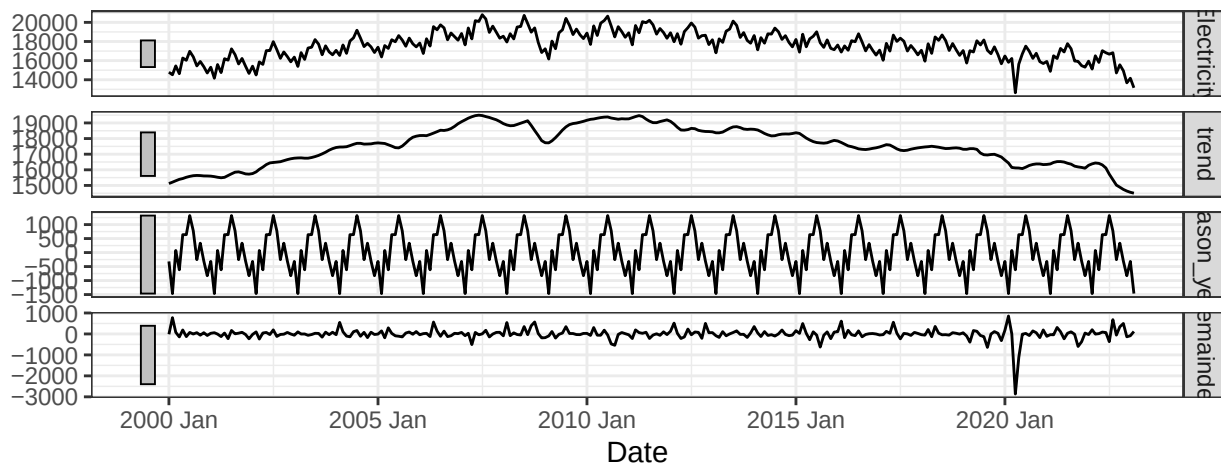
N0- The remainder component is white noise N1- The remainder component is not white noise

```
elec_stl_fit <- elec.train %>%
model(stl = STL(Electricity ~ trend(window = 7) + season(window = "periodic"),
robust = TRUE))

elec_stl_fit %>%
components() %>%
autoplot() + labs(title = "Robust STL Decomposition (Trend=7, Seasonal=Periodic)")
```

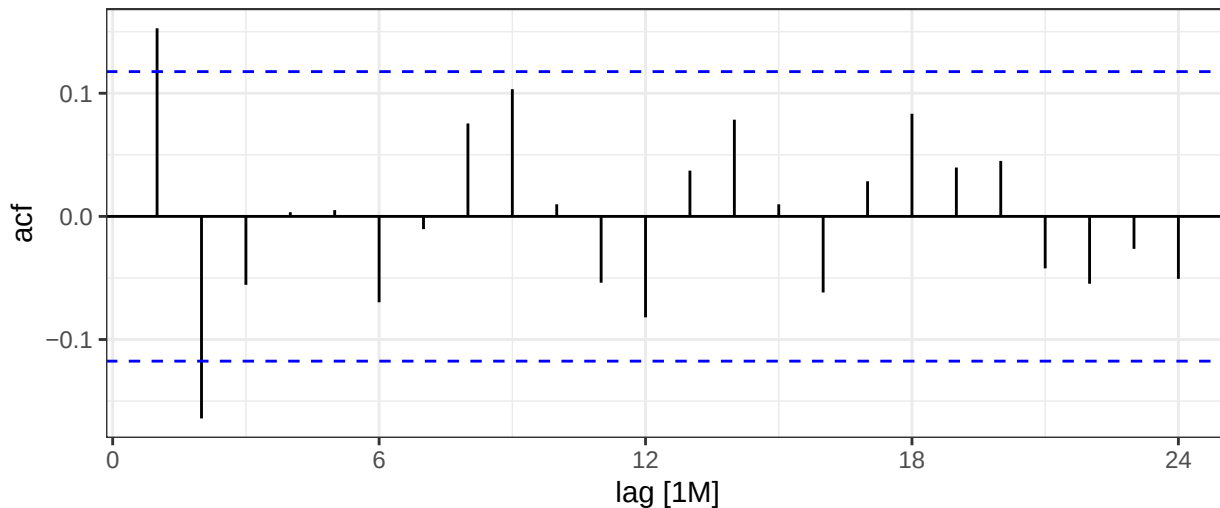
Robust STL Decomposition (Trend=7, Seasonal=Periodic)

Electricity = trend + season_year + remainder



```
elec_stl_fit %>%
components() %>%
ACF(remainder) %>%
autoplot() + labs(title = "ACF of the Remainder Component (STL)")
```

ACF of the Remainder Component (STL)



```
elec_stl_fit %>%
  components() %>% features(remainder, ljung_box, lag = 24)
```

```
## # A tibble: 1 x 3
##   .model lb_stat lb_pvalue
##   <chr>    <dbl>    <dbl>
## 1 stl      33.3      0.0984
```

- h. [5 points] From the exercise above, you should have landed on a single model having passed the white noise test at the 5% level of significance. Now, Mr. Fuentes would like to identify the best forecasting method.
- (i) In a single `model()` function use the `decomposition_model()` command to estimate the following candidate models for the **seasonally-adjusted** component:
- an ARIMA model (let R choose the best model based on the AICc criterion)
 - A Naive model
 - A mean model

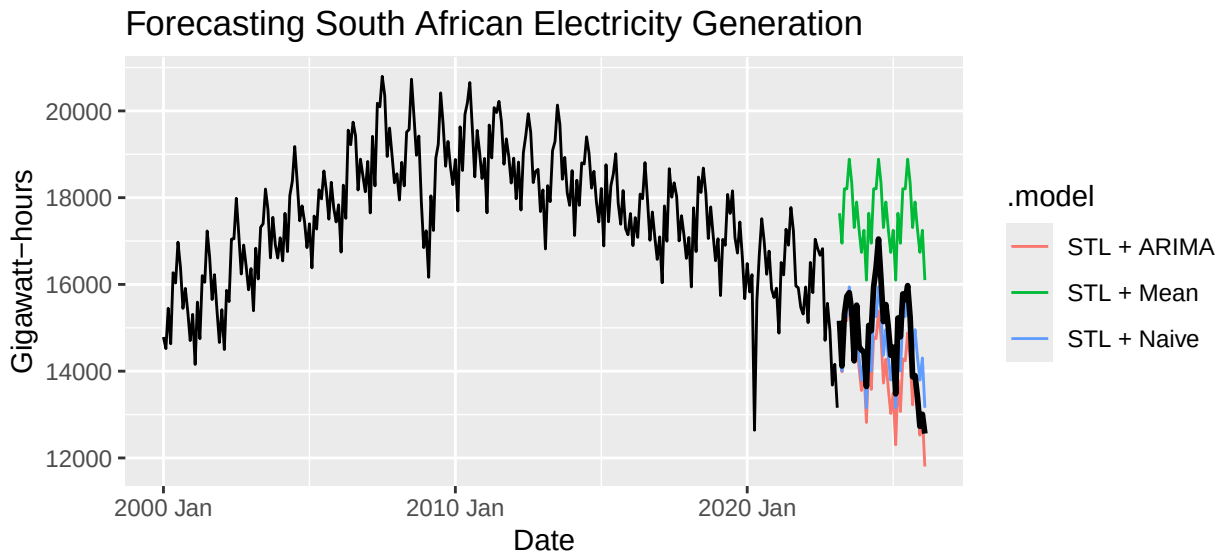
Note: Ensure that all models are properly named. Store the model results in this step.

```
elec_models <- elec.train %>%
  model(`STL + ARIMA` = decomposition_model(
    STL(Electricity ~ trend(window = 7) + season(window = "periodic"), robust = TRUE),
    ARIMA(season_adjust)),
  `STL + Naive` = decomposition_model(
    STL(Electricity ~ trend(window = 7) + season(window = "periodic"), robust = TRUE),
    NAIVE(season_adjust)),
  `STL + Mean` = decomposition_model(
    STL(Electricity ~ trend(window = 7) + season(window = "periodic"), robust = TRUE),
    MEAN(season_adjust)))
```

- (ii) Next, use your models above to produce forecasts over the test period. Present a single plot of the forecasts, **ensuring that the test data is included in the plot.**

```
elec_fc <- elec_models %>%
  forecast(h = "3 years")

elec_fc %>%
  autoplot(elec.train, level = NULL) +
  autolayer(elec.test, Electricity, color = "black", linewidth = 1) +
  labs( title = "Forecasting South African Electricity Generation",
        y = "Gigawatt-hours",
        x = "Date") + theme_gray()
```



(iii) Finally, use the `accuracy()` function to produce a table of the accuracy statistics for each model. Please speak to the following points:

- Which model is preferred under the RMSE and MAPE criteria? **Please arrange your table results on the RMSE column.** The STL + NAIVE
- What does the sign on the ME statistic tell you about each model's bias? Which model appears to be most biased towards over-predicting the data? How about towards under-prediction? The STL + Naive and ARIMA models were underpredicting while the STL+Mean model was way overpredicting.

Please restrict your table to only these three (3) statistics.

```
accuracy_table <- elec_fc %>%
  accuracy(elec.test) %>%
  # Select only the requested statistics
  dplyr::select(.model, RMSE, MAPE, ME) %>%
  # Arrange by RMSE (lowest to highest)
  arrange(RMSE)

print(accuracy_table)
```

```
## # A tibble: 3 x 4
##   .model      RMSE  MAPE    ME
##   <chr>      <dbl> <dbl> <dbl>
## 1 STL + Naive  639.   3.69  214.
## 2 STL + ARIMA 1005.   5.64  847.
## 3 STL + Mean  2795.  18.7 -2729.
```

Question 2: Forecasting HomeFront Realty’s Monthly Closings [30 points]

You’re in the technical round of a job interview at HomeFront Realty, a regional residential brokerage. The hiring manager has handed you a monthly series of closed transactions (homes sold) from January 2000 through December 2024 – it’s in `homefront_closings.csv`, in the `Exam` folder on GitHub – and asked you to walk her through how you’d build a forecast.

Sitting in on the panel is Kendyl, the firm’s Inspector of the Variance Transformation Taskforce, who looks at the time plot, raises an eyebrow, and observes that “the variance might need stabilizing.” She does not elaborate. The hiring manager turns back to you expectantly.

The interview is open-ended: there is no checklist on their side. They want to see how you think.

You decide to deliver a written walkthrough that the panel can read alongside your code. The audience is a *business reader who knows what a forecast is but not what an ARIMA is* – so plots and plain-English reasoning carry more weight than test statistics in isolation. **Submissions that read as code chunks with one-line comments will lose points.** The bullets below are the *minimum* the panel expects you to address, not a script to follow line by line.

A few constraints they have given you up front. Import the data and declare it as a `yearmonth`-indexed `tsibble`. Use December 2024 as the end of the sample. Your final forecast should be on the **original closings scale** – whatever transformations you apply along the way, the plot and subsequent forecasts, you hand the panel has to read in the units they care about (number of homes closed per month).

Your walkthrough should:

- Open with an `autoplot()` and a `gg_subseries()` of the series and a discussion of what they show. Are some months systematically higher or lower than others? Does the variability of monthly closings look stable across the sample, or does it shift over time (aka does seasonal variations appear to be increasing over time)? The panel wants to know whether you can *see* the data before you model it.

March, July, and August seem to be the month with the highest closing numbers. While the months like January, December, and November seem to be the lowest. The variations definitely do not seem stable over time as they keep getting larger and larger over time compared to the starting area’s variance.

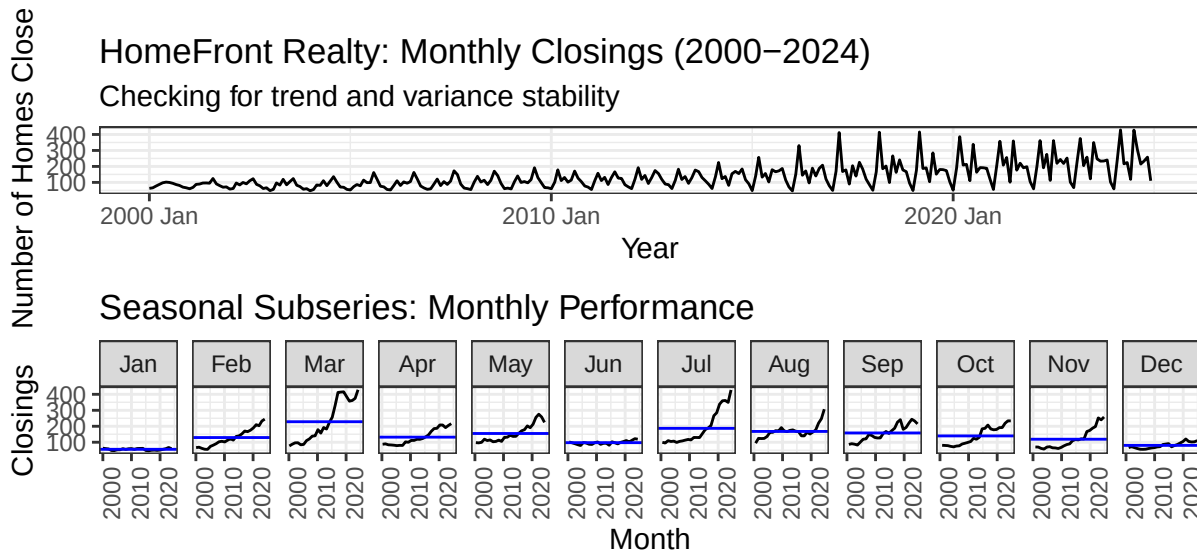
```
url2 <- "https://raw.githubusercontent.com/Shamar-Stewart/Forecasting-S26/refs/heads/main/Data/homefront_cl
raw_data1 <- read_csv(url2)

homefront_ts <- raw_data1 %>%
  mutate(Month = yearmonth(index)) %>%
  as_tsibble(index = Month) %>%
  filter(Month <= yearmonth("2024 Dec"))

plot1 <- homefront_ts %>%
  autoplot(value) +
  labs(title = "HomeFront Realty: Monthly Closings (2000-2024)",
       subtitle = "Checking for trend and variance stability",
       y = "Number of Homes Closed",
       x = "Year")

plot2 <- homefront_ts %>%
  gg_subseries(value) +
  labs(title = "Seasonal Subseries: Monthly Performance",
       y = "Closings",
       x = "Month")

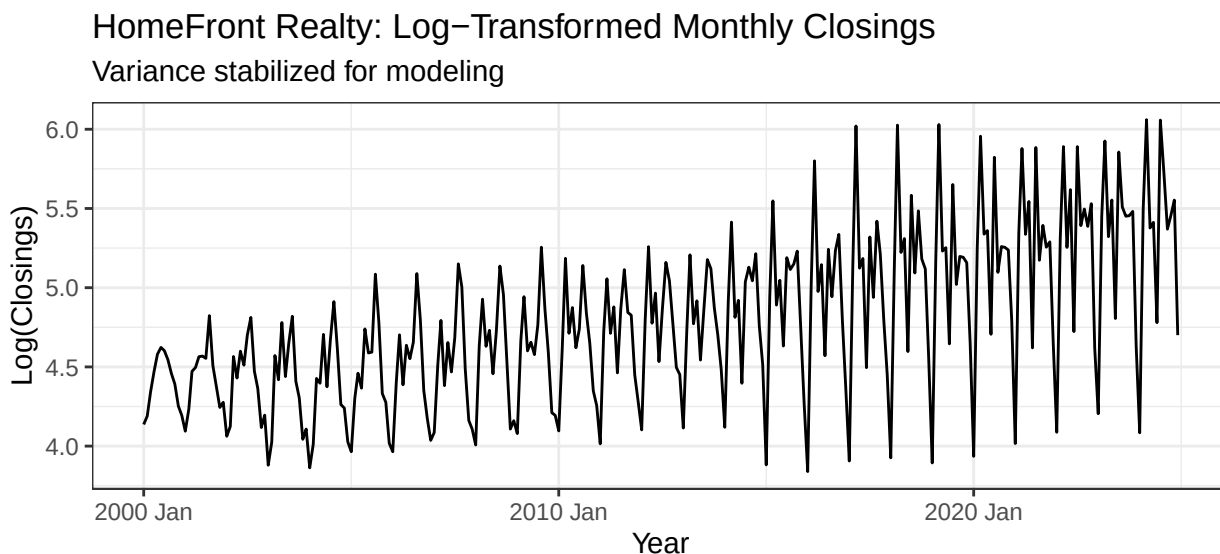
plot1/plot2
```



- Engage with **Kendyl's variance observation**. Decide whether you agree with her – if you do, apply a **log** transformation, plot the result, and explain in plain English what the transformation has accomplished and why it matters for the model that follows. If you disagree, defend that call. Either way, your final forecast must come back out on the original closings scale.

I back up Kendyl's idea of applying a log transformation. By establishing a logarithmic transformation, we turned the market's growing swinging variance into a subtle yet more controlled pattern that the model can actually understand. This was crucial because it prevents the large sales spikes from skewing our models and ensures our predictions aren't skewed by the scale of any modern growth. Thus, this extra step leads to a much more reliable forecast that stays realistic even during the market's most busy times.

```
homefront_ts %>%
  autoplot(log(value)) + labs(title = "HomeFront Realty: Log-Transformed Monthly Closings",
    subtitle = "Variance stabilized for modeling",
    y = "Log(Closings)",
    x = "Year")
```



- Assess **stationarity**, formally and visually. State the null and your verdict at the 5% level. If differencing is required, justify the kind you choose (regular vs. seasonal, or both).

Assessing at the 5% alpha level with a p-value of .01 and with it being less than the alpha value, we can reject the null hypothesis. We can conclude that this is not a stationary process. I will run both differences to get rid of any trend and any seasonality.

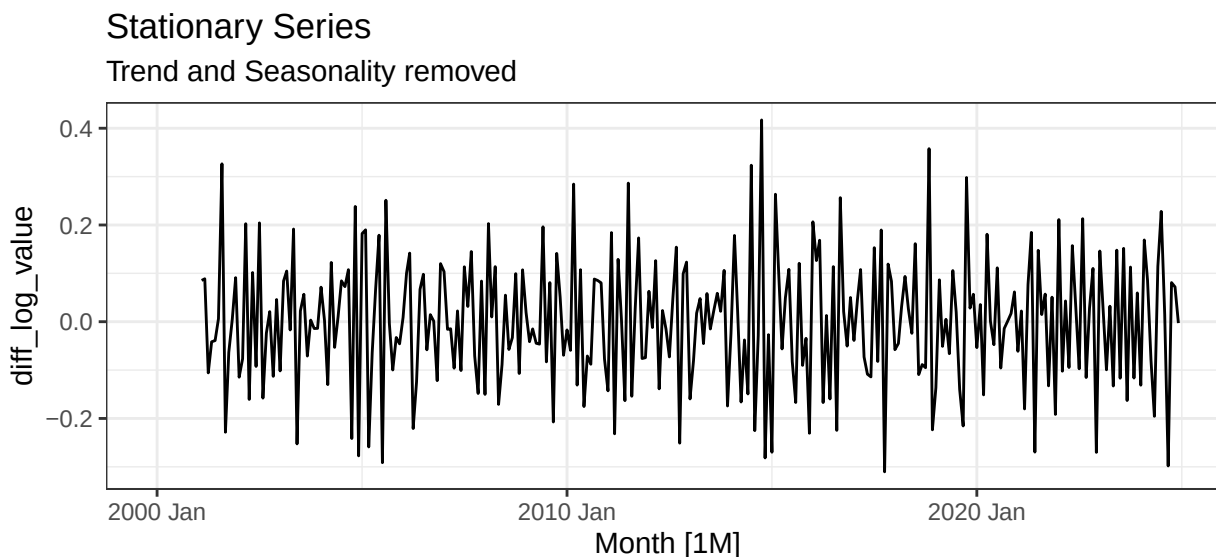
H0- Data is a stationary process H1- Data is not a stationary process

```
homefront_ts %>%
  mutate(log_value = log(value)) %>%
  features(log_value, unitroot_kpss)
```

```
## # A tibble: 1 x 2
##   kpss_stat kpss_pvalue
##   <dbl>      <dbl>
## 1      4.24        0.01
```

```
homefront_ts_stationary <- homefront_ts %>%
  mutate(diff_log_value = difference(log(value), 12) %>% difference(1))

homefront_ts_stationary %>%
  autoplot(diff_log_value) +
  labs(title = "Stationary Series", subtitle = "Trend and Seasonality removed")
```

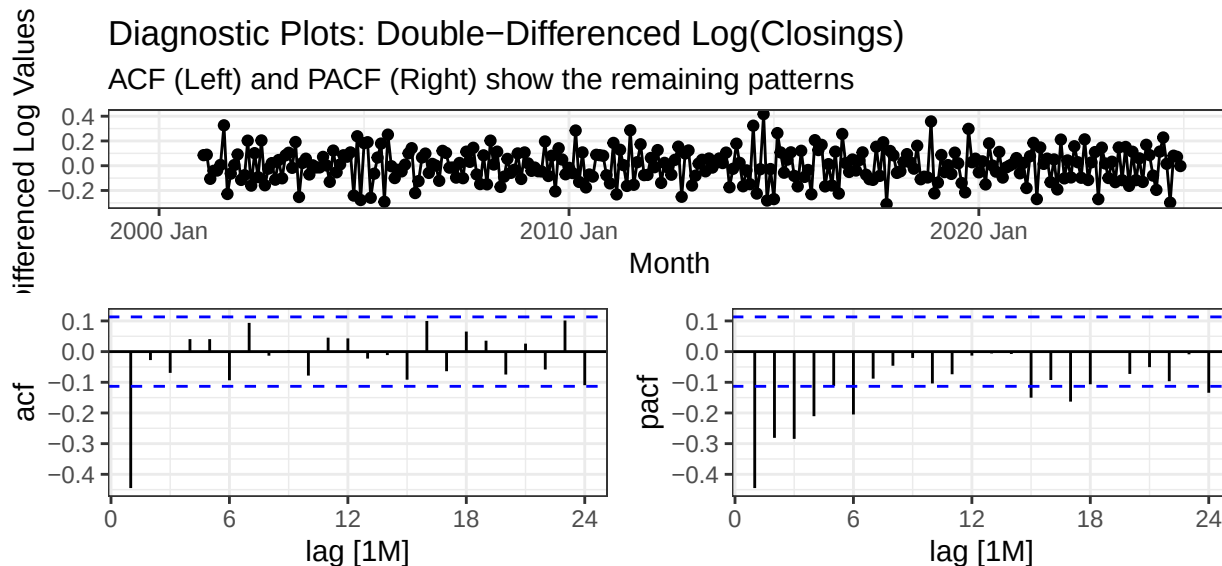


- Use a `gg_tsdisplay()` of the stationary series to identify candidate ARIMA models. Walk the panel through how you read the ACF and PACF to land on a small set of candidates, and **manually fit the candidates** so an information criterion can arbitrate between them. Explain why your preferred model wins.

We basically just look at any significant lag spikes in the PACF and see that the first few are out of the band with a couple of other later on in the plot. While in the ACF the first lag is the only significant one leading us to see that our seasonal differencing worked.

The auto search model did the best with the lowest AICc criteria value being -505. The `auto_search` model won because it provided the most accurate explanation of past performance using the fewest possible mathematical parts. While our manual candidates like hybrid or shock were close in the running. The algorithm found a slightly more efficient model that balances these two goals better than other models

```
homefront_ts %>%
  mutate(diff_log_value = log(value) %>% difference(12) %>% difference(1)) %>%
  gg_tsdisplay(diff_log_value, plot_type = "partial") + labs(title = "Diagnostic Plots: Double-Differenced Log",
    subtitle = "ACF (Left) and PACF (Right) show the remaining patterns",
    y = "Differenced Log Values")
```



```
fit_candidates <- homefront_ts %>%
  model(
    momentum_ar1 = ARIMA(log(value) ~ pdq(1,1,0) + PDQ(0,1,0)),
    shock_ma1    = ARIMA(log(value) ~ pdq(0,1,1) + PDQ(0,1,0)),
    hybrid_arima1 = ARIMA(log(value) ~ pdq(1,1,1) + PDQ(0,1,0)),
    auto_search  = ARIMA(log(value)))

fit_candidates %>%
  glance() %>%
  dplyr::select(.model, AICc) %>%
  arrange(AICc)
```

```
## # A tibble: 4 x 2
##   .model      AICc
##   <chr>      <dbl>
## 1 auto_search -505.
## 2 hybrid_arima1 -498.
## 3 shock_ma1    -497.
## 4 momentum_ar1 -401.
```

- **Estimate** your preferred model, report the fit, and run residual diagnostics – including a Ljung-Box test with appropriate lags and degrees of freedom. State the null and your conclusion. If the residuals fail any of the standard checks, say so explicitly and explain whether the failure is fatal or something you can work around.

N0- The residuals are white noise N1- The residuals are not white noise

Evaluating the p-value of .404 at the .05 alpha level with the p-value being higher we can rule that we fail to reject the null. Thus, leading to our conclusion that this is a white noise/random process and that we have extrapolated sufficient data from the series.

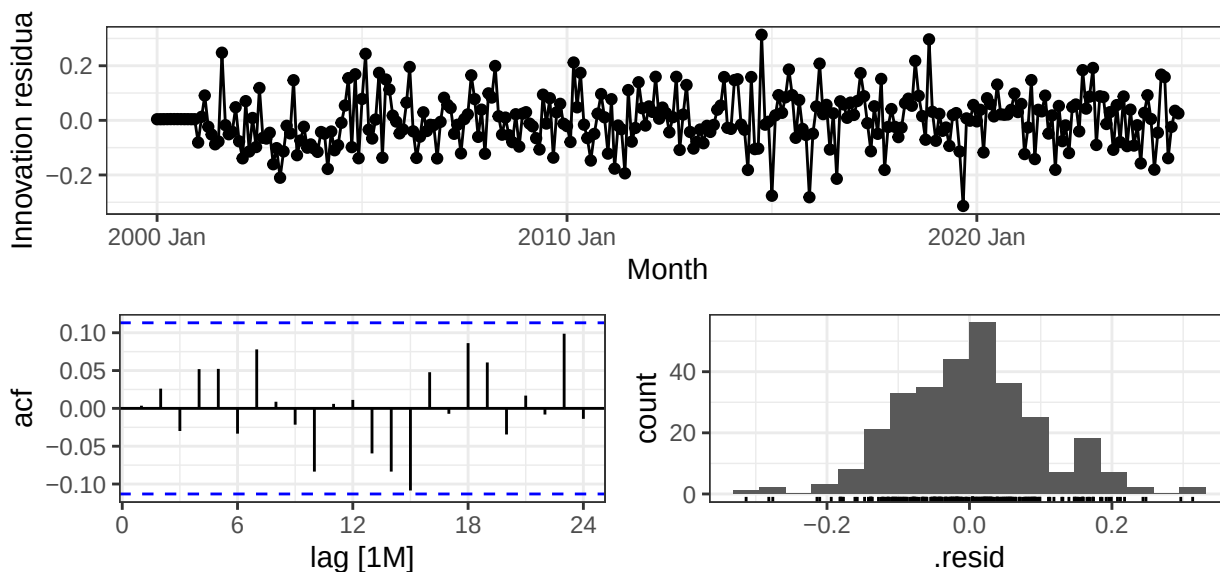
```
final_fit <- homefront_ts %>%
  model(BestModel = ARIMA(log(value)))

report(final_fit)
```

```
## Series: value
## Model: ARIMA(0,0,1)(0,1,0)[12] w/ drift
## Transformation: log(value)
```

```
##
## Coefficients:
##      mal      constant
##      0.1141      0.0392
## s.e.  0.0566      0.0065
##
## sigma^2 estimated as 0.009998: log likelihood=256
## AIC=-505   AICc=-505   BIC=-494
```

```
final_fit %>% gg_tsresiduals()
```



```
final_fit %>%
  augment() %>%
  features(.innov, lbjung_box, lag = 24, dof = 3)
```

```
## # A tibble: 1 x 3
##   .model    lb_stat lb_pvalue
##   <chr>      <dbl>   <dbl>
## 1 BestModel    21.9     0.404
```

- **Forecast 3 years ahead** with a 90% prediction interval, plotted along with the historical series. Comment on the path of the forecast, the width of the interval, and what the interval implies about how confident the panel should be in next year's staffing and inventory plans.

You're trying to land this job. Show them that you understand both the mechanics and the judgment calls.

The central line keeps up with an upward trend as the series suggests and keeps the seasonality of the business as well. You can see that the "fan" or confidence intervals start off tight and expand more later in the future, this just shows the accuracy possible in the beginning of the forecast with a bit more uncertainty due to extraneous reasons that could affect the forecast later down the line. This model gives a bright look on the future with realistic expectation and certain risks or uncertainties kept in mind. Although respecting the prior data and taking all the information we could squeeze out of it to best forecast what is to come for the company.

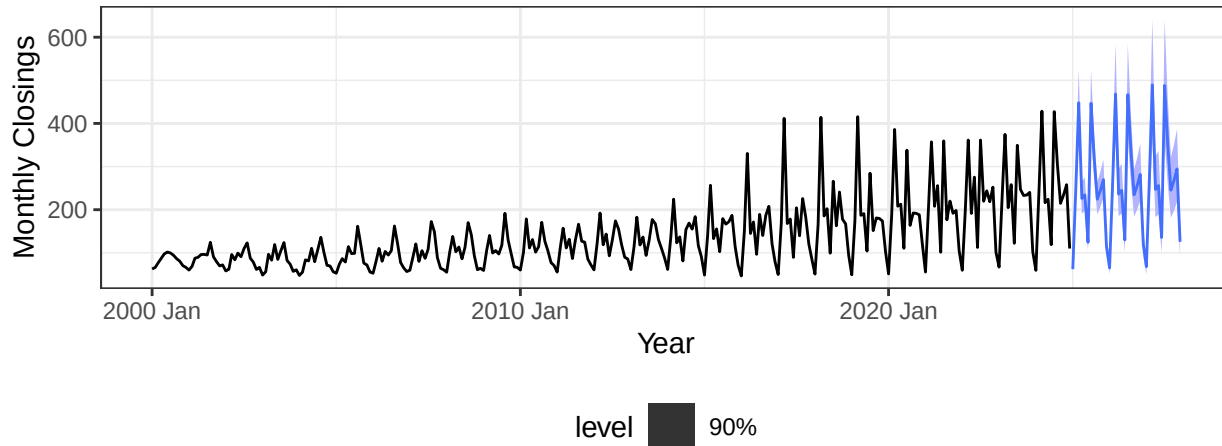
```
final_fit %>%
  forecast(h = "3 years") %>%
  autoplot(homefront_ts, level = 90) + labs(title = "HomeFront 3-Year Closing Forecast (2025-2027)",
  subtitle = "90% confidence",
```



```
y = "Monthly Closings",  
x = "Year"  
)
```

HomeFront 3-Year Closing Forecast (2025–2027)

90% confidence



Question 3: The ARMA model [Total: 40 points]

Building permits are a closely watched **leading economic indicator** – they signal future construction activity before the first foundation is poured – and they feed directly into infrastructure planning, budget projections, and housing policy at the state level.

Alex A. has just landed a job on the modelling team of a government agency tasked with forecasting building permits in Alabama. His first deliverable is a 2-year forecast of the monthly **Seasonally Adjusted New Private Housing Units Authorized by Building Permits in Alabama** (FRED symbol: ALBPPRIVSA), sampled from 1988M01 through 2019M01. The series is stored in ALBPPRIVSA.csv in our GitHub Data folder.

A quick note on what we're doing and why. The textbook Box-Jenkins workflow handles a nonstationary series by taking d differences until the series looks stationary, fitting an $\text{ARMA}(p, q)$ to the differenced data, and calling the combined object an $\text{ARIMA}(p, d, q)$. An ARMA model is just the special case where $d = 0$ – equivalently, an $\text{ARMA}(p, q)$ is an $\text{ARIMA}(p, 0, q)$. The real modeling choice in front of us is therefore not “ARMA vs. ARIMA” but **whether to difference at all** before fitting.

For this exercise we are going to skip the differencing step and fit ARMA models directly to the **levels** of the series. This is a defensible shortcut when the goal is short-horizon forecasting and the nonstationarity is mild; it is *not* a substitute for proper unit-root testing in serious applied work. **We will also skip unit-root testing entirely here** and proceed directly to the ACF and PACF of the levels to identify candidate models. Your candidate models in parts (b) and (d) should therefore be specified as ARMA models on the levels – which, in R, you will fit as $\text{ARIMA}(p, 0, q)$.

- a. [5 points] Import the data, declare the date variable as a `yearmonth` object, and convert it to a `tsibble`. Provide a time-series plot and briefly comment on its key features (trend, irregular variation). **Since the data are seasonally adjusted, you do not need to comment on seasonality.**

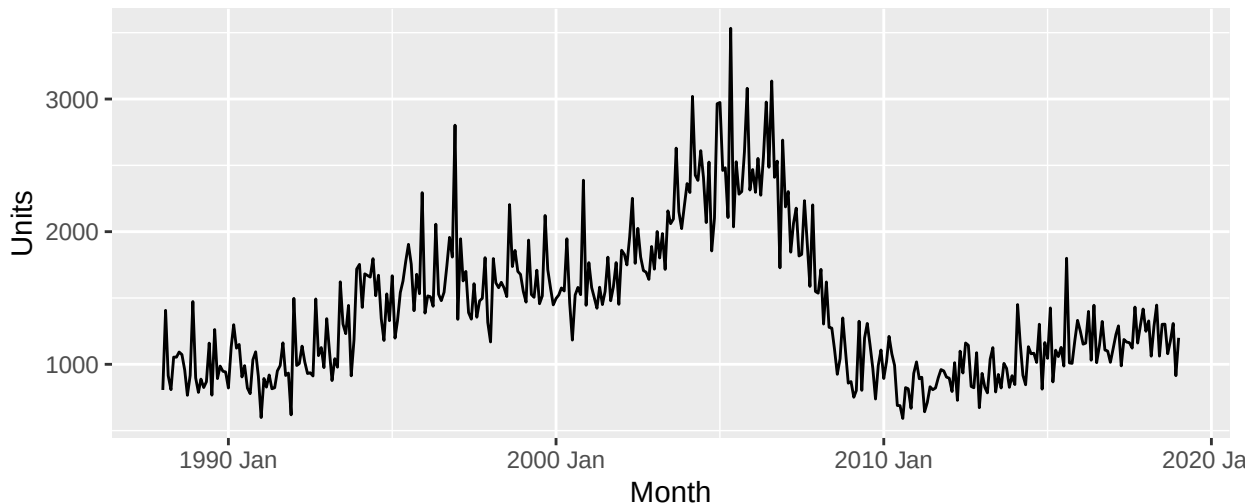
The plot shows higher variability until a structural break/change in level around the 2008-2009 years with a seemingly lower variance from then on. It also shows an upward trend until 2008 where a large decline occurs with a small returning upward trend relative to the decline's minimal level.

```
url3 <- "https://raw.githubusercontent.com/Shamar-Stewart/Forecasting-S26/refs/heads/main/Data/ALBPPRIVSA.csv"
raw_data2 <- read_csv(url3)

al_permits <- raw_data2 %>%
  mutate(
    DATE = yearmonth(DATE)
  ) %>%
  as_tsibble(
    index = DATE
  ) %>%
  filter(
    DATE >= yearmonth("1988 Jan") & DATE <= yearmonth("2019 Jan")
  )

al_permits %>%
  autoplot(
    ALBPPRIVSA,
    labs(
      title = "New Private Housing Units Authorized by Building Permits (Alabama)",
      y = "Units",
      x = "Month"
    ) + theme_gray()
  )
```

New Private Housing Units Authorized by Building Permits (Alabama)

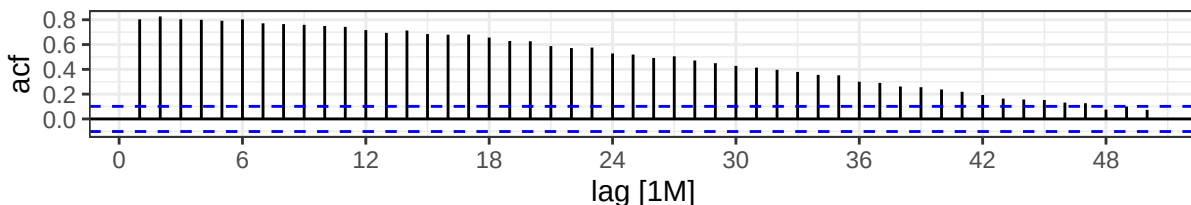


- b. [5 points] Use the `ACF()` and `PACF()` functions to plot the **ACF and PACF with `lag_max = 50`** for the **levels** of the series, and use them to identify candidate **ARMA** model(s). Present the plots as a 2×1 grid.

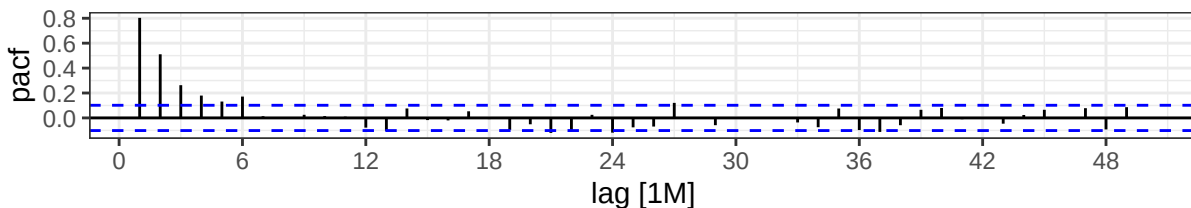
```
p1 <- al_permits %>%
  ACF(ALBPPRIVSA, lag_max = 50) %>%
  autoplot() + labs(title = "ACF: ALBPPRIVSA (Levels)")
p2 <- al_permits %>%
  PACF(ALBPPRIVSA, lag_max = 50) %>%
  autoplot() + labs(title = "PACF: ALBPPRIVSA (Levels)")

p1 / p2
```

ACF: ALBPPRIVSA (Levels)



PACF: ALBPPRIVSA (Levels)



- Briefly justify your choice of model(s) in **plain English** – imagine you're explaining your reasoning to Alex's manager, who knows time-series basics but doesn't want a textbook recap.

Reminder: the data are seasonally adjusted, so there is no seasonal element to model.

I would suggest using an ARMA(2,0) model further on. It's a easy but robust choice that captures the influence of the last two months of permit data while evaluating for the ACF's long-term momentum.

An ARMA(1,1) is another option available. The AR1 captures the momentum, and the moving average term can help smooth out the dramatic variance in spikes that we see in permit data.

- c. [5 points] Write out the econometric equation(s) for the model(s) you chose in (b) using **general notation**. Include an intercept term in each equation.

```
fit <- al_permits %>%
model(arma20 = ARIMA(ALBPPRIVSA ~ 1 + pdq(2,0,0)),
arma11 = ARIMA(ALBPPRIVSA ~ 1 + pdq(1,0,1)))
```

```
fit %>%
  dplyr::select(arma20) %>%
  report()
```

```
## Series: ALBPPRIVSA
## Model: ARIMA(2,0,0) w/ mean
##
## Coefficients:
##          ar1      ar2  constant
##          0.3894  0.5137      135.5
## s.e.    0.0445  0.0445      13.6
##
## sigma^2 estimated as 75178:  log likelihood=-2623
## AIC=5253   AICc=5253   BIC=5269
```

```
fit %>%
  dplyr::select(arma11) %>%
  report()
```

```
## Series: ALBPPRIVSA
## Model: ARIMA(1,0,1) w/ mean
##
## Coefficients:
##          ar1      ma1  constant
##          0.9896 -0.7411      13.79
## s.e.    0.0066  0.0304      2.83
##
## sigma^2 estimated as 65973:  log likelihood=-2598
## AIC=5205   AICc=5205   BIC=5220
```

- d. [5 points] We are going to compare your candidate from (b) against three alternatives. Consider:

- A restricted version of the **AR(6) model**, denoted $AR(6)^*$, in which the **fifth lag is dropped**.
- Two simple benchmarks: $AR(1)$ and $AR(2)$.

Write out the econometric equations for all three models in general notation, with intercepts included.

```
fit_alternatives <- al_permits %>%
model(ar6_restricted = ARIMA(ALBPPRIVSA ~ 1 + pdq(6,0,0) + fixed(list(ar5 = 0))),
ar1_benchmark = ARIMA(ALBPPRIVSA ~ 1 + pdq(1,0,0)),
ar2_benchmark = ARIMA(ALBPPRIVSA ~ 1 + pdq(2,0,0)))
```

- e. [10 points] Using the **full sample period**, estimate every model from parts (b) and (d). Give each model a clear, descriptive name – you'll be calling them by name in (f) and (g).
- Use `tidy()` to extract the parameter estimates for each model. Discuss which coefficients are statistically significant and what they say about the dynamics of the building permits series.

- Report the **AIC** and **BIC** for all four models in a clean `kable()` table with labeled rows and columns. **State which model each criterion prefers and why.**

AIC prefers the AR6_Restricted model as it has the lowest AIC criterion value and the BIC prefers the ARMA11 model due to it also being the lowest BIC criterion value.

```
fit_all <- al_permits %>%
  model(
    ARMA11 = ARIMA(ALBPPRIVSA ~ 1 + pdq(1,0,1) + PDQ(0,0,0)),
    AR1     = ARIMA(ALBPPRIVSA ~ 1 + pdq(1,0,0) + PDQ(0,0,0)),
    AR2     = ARIMA(ALBPPRIVSA ~ 1 + pdq(2,0,0) + PDQ(0,0,0)),
    AR6_Restricted = ARIMA(ALBPPRIVSA ~ 1 + pdq(6,0,0, fixed = list(ar5 = 0)) +
                          PDQ(0,0,0), transform.pars = FALSE))

fit_all %>% tidy()
```

```
## # A tibble: 15 x 6
##   .model      term      estimate std.error statistic    p.value
##   <chr>      <chr>      <dbl>    <dbl>    <dbl>    <dbl>
## 1 ARMA11     ar1         0.990   0.00662   149.      0
## 2 ARMA11     ma1        -0.741   0.0304   -24.4    2.57e-79
## 3 ARMA11     constant    13.8     2.83     4.87    1.67e- 6
## 4 AR1        ar1         0.804   0.0306    26.3    5.55e-87
## 5 AR1        constant   277.     16.3     17.0    3.00e-48
## 6 AR2        ar1         0.389   0.0445     8.76    6.93e-17
## 7 AR2        ar2         0.514   0.0445    11.5    1.45e-26
## 8 AR2        constant   135.     13.6     9.94    8.18e-21
## 9 AR6_Restricted ar1         0.170   0.0505     3.37    8.32e- 4
## 10 AR6_Restricted ar2         0.291   0.0510     5.71    2.34e- 8
## 11 AR6_Restricted ar3         0.172   0.0511     3.37    8.27e- 4
## 12 AR6_Restricted ar4         0.120   0.0529     2.27    2.38e- 2
## 13 AR6_Restricted ar5          0      0          NaN     NaN
## 14 AR6_Restricted ar6         0.198   0.0505     3.93    1.01e- 4
## 15 AR6_Restricted constant    64.4    11.5     5.60    4.25e- 8
```

```
fit_all %>%
  glance() %>%
  dplyr::select(.model, AIC, BIC) %>%
  knitr::kable(col.names = c("Model", "AIC", "BIC"),
    caption = "Model Selection Criteria",
    digits = 2)
```

Table 1: Model Selection Criteria

Model	AIC	BIC
ARMA11	5205	5220
AR1	5364	5376
AR2	5253	5269
AR6_Restricted	5203	5231

Practical hints for fitting these in R:

- Include an intercept in every model (the `~ 1` piece of the formula).

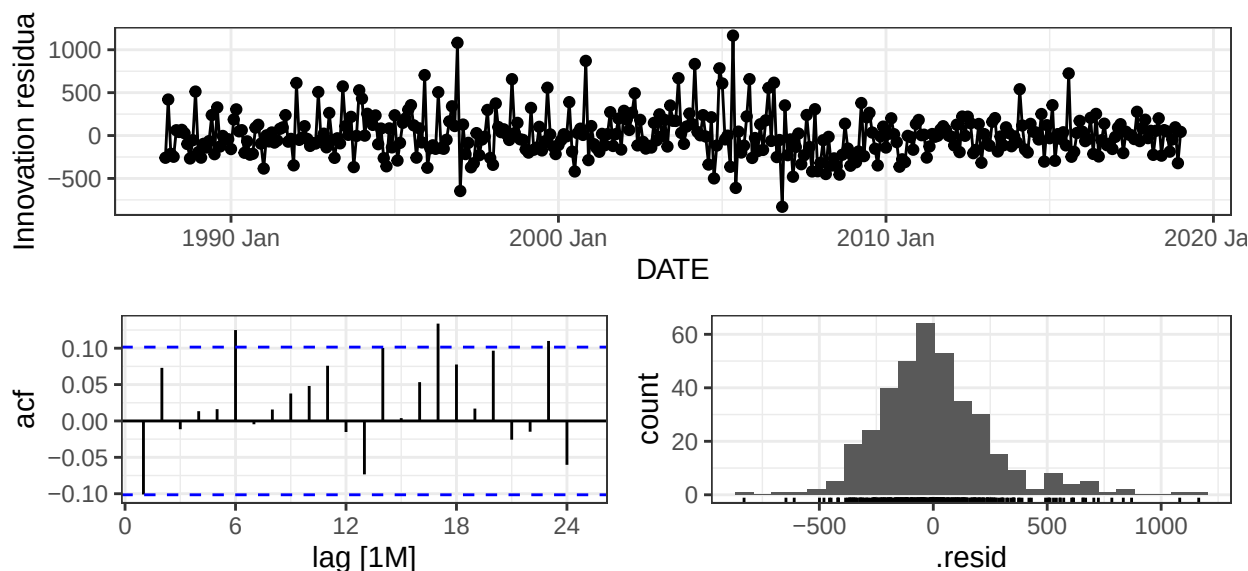
- (ii) Because $\text{ARMA}(p, q) \equiv \text{ARIMA}(p, 0, q)$, an $\text{AR}(4)$ is fit with `ARIMA(... ~ pdq(4,0,0) + ...)`. **This is *not* the same as `pdq(1:4,0,0)`**, which tells R to *search* across lags 1 through 4 and pick its own preferred fit – precisely what we are avoiding here.
- (iii) Since the data are seasonally adjusted, include `PDQ(0,0,0)` in every model spec to stop R from layering a seasonal structure on top of yours.
- (iv) The $\text{AR}(6)^*$ model needs two extra ingredients to enforce the zero-restriction on the 5th lag:
 - pass `fixed = list(ar5 = 0)` *inside* `pdq()` to pin the 5th-lag coefficient to zero, and
 - pass `transform.pars = FALSE` directly to `ARIMA()` so R doesn't quietly reparameterize the model and undo your restriction.

Feel free to consult Google if any of this is fuzzy.

- f. [5 points] Run **residual diagnostics** on your **BIC-preferred model** from (e) and comment on whether the model looks adequate. You should:
 - Produce a residual plot using `gg_tsresiduals()`.
 - Report and interpret a **Ljung-Box test** with a **lag order of 24 (2 years)**, using the **appropriate degrees of freedom** for the model. State the null and alternative explicitly, and give your conclusion at the 5% level.

We will use the .05 alpha value to evaluate the p-value of .00492. Since the p-value is lower than the alpha value we can reject the null hypothesis. Thus, concluding that there is enough evidence to reject the null and state that the residuals are not white noise. H0- The residuals are white noise H1- The residuals are not white noise

```
fit_all %>%
  dplyr::select(ARMA11) %>%
  gg_tsresiduals()
```



```
fit_all %>%
  dplyr::select(ARMA11) %>%
  augment() %>%
  features(.innov, ljung_box, lag = 24, dof = 2)
```

```
## # A tibble: 1 x 3
##   .model lb_stat lb_pvalue
##   <chr>    <dbl>    <dbl>
## 1 ARMA11    42.9    0.00492
```

- g. [5 points] Using the **BIC-preferred model**, produce Alex's **2-year forecast** of building permits. Plot the forecast on top of the historical series and comment on what you see. **Pay particular attention to the width of the forecast intervals** – what do they say about how confident Alex should be when he hands this over?

The line is pretty straight with a slight upward trend too. I see a very wide interval of confidence at both respective levels. It just portrays a too wide of an interval that isn't too confident on more accurate values and should be wary about trusting the model.

```
forecast_arma11 <- fit_all %>%  
  dplyr::select(ARMA11) %>%  
  forecast(h = "2 years")  
  
forecast_arma11 %>%  
  autoplot(al_permits %>% filter(DATE >= yearmonth("2010 Jan"))) +  
  labs(title = "2-Year Forecast of Alabama Building Permits",  
        subtitle = "Model: ARMA(1,1) [BIC-Preferred]",  
        y = "Units",  
        x = "Month") +  
  theme_minimal()
```

